Virtualizing a Cloud Data Infrastructure with BRAD

Geoffrey X. Yu
Massachusetts Institute of Technology
geoffxy@mit.edu

Over the past decade, the cloud has transformed how organizations manage their data through two key forces: (i) offering a wealth of specialized database engines optimized for diverse workloads, and (ii) enabling "one-click" on-demand access to conceptually "infinite" resources. To fully reap these benefits, organizations are forced to curate a collection of specialized database engines, each chosen for the performance, functionality, and/or cost benefits they provide on different parts of their workload. For example, a company might use Aurora to manage their clients' accounts with transactions, Snowflake to analyze historical sales data, and BigQuery Omni for exploratory analysis. Benefits aside, such multi-system infrastructures also introduce new management challenges. Data engineers must (i) choose a suitable set of engines (out of dozens) for their workload, (ii) decide how to partition and/or replicate their data across the engines, (iii) figure out which engines to use for each aspect of their workload (e.g., which queries go to each engine), (iv) provision the engines appropriately, and (v) repeat these steps each time their workload or business needs change. Navigating these decisions requires data engineers to address two (often conflated) concerns. First, they must understand the functionality, performance, and data freshness requirements of the end-user applications that will use the infrastructure. For example, an e-commerce application probably needs transactional access to its data with high isolation guarantees and at a low latency. In contrast, a data scientist conducting exploratory analysis over historical sales records might only need access to reasonably fresh snapshots of data and would likely be issuing complex long-running analytical queries. Second, they then must decide how to *realize* these various requirements on physical cloud engines while minimizing costs. This second concern is especially challenging; prior work showed that an optimal infrastructure depends on many interconnected factors such as the selectivity of queries, service level objectives (SLOs), and dynamic load of the system [3]. Consequently, many organizations struggle to design their infrastructure while also keeping costs under control [2].

To address these challenges, we propose BRAD: a new cloud-native database and infrastructure management system that automatically designs and operates a data infrastructure to optimize for performance and cost [3]. The key innovation in BRAD is that it *virtualizes* the cloud data infrastructure. Users define a set of *virtual database engines* (VDBEs), which capture the "application-level requirements" that an end-user application expects from the engine. BRAD then automatically decides how to best realize these VDBEs onto a set of concrete physical database engines (e.g., Aurora, Redshift, Snowflake, etc.) by solving a cost-based optimization problem leveraging learned performance models. Operationally, BRAD is a proxy-like indirection layer. It exposes one endpoint per VDBE. Clients connect to one of these VDBE endpoints and BRAD forwards their queries to a physical engine for execution (Figure 1).

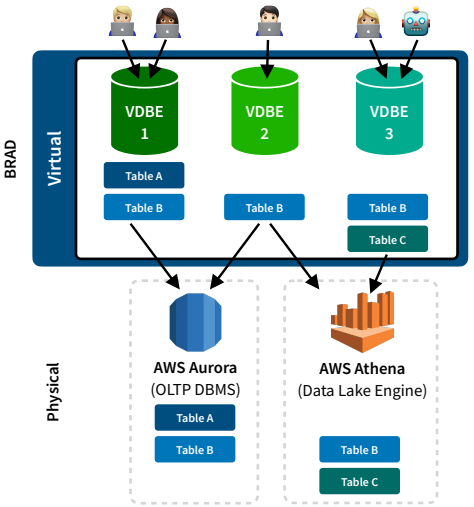

Figure 1: An overview of BRAD's system architecture. Here it realizes three VDBEs using two physical engines (Aurora and Athena).

Virtual Database Engines. Concretely, users define a virtual database engine by specifying (i) a set of tables (with their schemas), (ii) a freshness constraint (explained next), and (iii) an optional performance constraint (e.g., queries run under 30 seconds). The key twist is that VDBE table sets *do not* have to be disjoint. If a client, connected to any VDBE, writes to table T and T is also in VDBE V, V's freshness constraint specifies how soon the write must be visible to clients connected to V. This can be "immediately," implying strict serializability, or with at most X seconds of delay (we always run queries against a snapshot). The power of this abstraction is that it makes data access requirements (which depend on the application) explicit, while maintaining flexibility on how they are realized. For example, BRAD is free to make replicas of a table and place them on multiple engines (thereby allowing multiple engines to support queries on the table), as long as it can maintain each VDBE's freshness requirements.

Example: Classic HTAP. Imagine we want to run transactions and analytics over the same schema. We want transactions to run under strict serializability, but are okay with running the analytics on a snapshot that is at most 10 minutes stale. We can express these requirements using two VDBEs: one with a freshness requirement of strict serializability and another with no more than 10 minutes stale. Both VDBEs contain the same set of tables. BRAD can realize these VDBEs by mapping them to one physical engine (e.g., Aurora), thus placing a single copy of all the tables there. This physical infrastructure is cost-optimal at small workload scales (i.e., it is not cost-effective to start a data warehouse). At larger workload scales, BRAD can place one copy of the tables on Aurora and another on Redshift and create a zero-ETL pipeline [1] between the two to quickly replicate Aurora writes onto Redshift. The transactional VDBE would map to Aurora and the analytical VDBE to Redshift.

Handling ETLs. Under the VDBE abstraction, ETLs work like in any other data infrastructure. For transformations that run on external tools (e.g., AWS Glue), the tool can simply read data from one VDBE and write the transformed data to the other. For ELT-based transformations (i.e., that run in an engine), users can define a VDBE that contains the source and destination tables for the transformation and run their DML statements on that VDBE. By realizing the VDBE on a physical engine, BRAD would automatically take care of scheduling the transformation queries on a physical engine.

Biography

Geoffrey is a PhD student in the Data Systems Group at MIT, advised by Tim Kraska. Geoffrey currently works on BRAD: a new system that automatically designs and operates multi-engine cloud data infrastructures—powered by learned cost-based optimization and a novel virtualized database engine abstraction. He is also broadly interested in cloud-native database management systems, systems-oriented database problems, and large-scale systems. Before starting his PhD, Geoffrey completed a master's degree in Computer Science at the University of Toronto where he worked on GPU performance prediction models to help deep learning researchers and practitioners select the best GPU for their training workloads. Geoffrey received his Bachelor in Software Engineering from the University of Waterloo. Geoffrey is a recipient of the NSERC Canada Graduate Scholarship – Doctoral (2020) and the Snap Research Scholarship (2019).

Website: www.geoffreyyu.com

References

[1] Amazon AWS. AWS announces Amazon Aurora zero-ETL integration with Amazon Redshift. `https://aws.amazon.com/about-aws/whats-new/2022/11/amazon-aurora-zero-etl-integration-redshift/`.

[2] Xingyu Gu, Adam Ronthal, Robert Thanaraj, and Julian Sun. Data and Analytics Cloud Adoption Survey Reveals Data Governance and Cost Challenges. Gartner Report, 2024. `https://www.gartner.com/document/5106731`.

[3] Tim Kraska, Tianyu Li, Samuel Madden, Markos Markakis, Amadou Ngom, Ziniu Wu, and Geoffrey X. Yu. Check Out the Big Brain on BRAD: Simplifying Cloud Data Processing with Learned Automated Data Meshes. *Proceedings of the VLDB Endowment*, 16(11):3293–3301, 8 2023.